A Framework for Synthetic Knowledge Graph Generation in Fraud Detection

Nasvin Nabeel
Rochester, United States
nl2168@rit.edu

***Abstract*—Graph-based fraud detection methods such as Graph Neural Networks (GNNs) can identify relational patterns that are often missed by traditional feature-based classifiers. However, progress in this area is frequently limited by restricted access to real graphs due to privacy, proprietary constraints, and the cost of reliable labels. This work addresses this challenge by building a schema-driven synthetic knowledge graph (KG) generation pipeline for fraud detection. Our approach uses a schema that specifies node types, relationship types, property definitions, and structural constraints, then generates a synthetic knowledge graph with configurable size and topology and corrupts it with controlled fraud patterns for Graph Neural Network training. The proposed pipeline is designed to be reusable and easily adapted to multiple application domains with minimal reconfiguration.**

I. Introduction

Fraud detection is a persistent challenge in modern digital platforms, including e-commerce, online banking, and payment networks [1]. The goal is to identify suspicious behavior early and accurately, often under constraints such as high transaction volume, evolving attacker strategies, and limited ground-truth labels [2]. Traditional approaches such as rule-based systems and tabular machine-learning models (e.g., logistic regression and gradient-boosted trees), typically treat each event or user record independently and rely heavily on engineered features [3]. While effective in some settings, these methods may fail to capture coordinated or multi-entity behavior where the evidence of fraud is primarily relational [3].

Graphs provide a natural representation for such environments by modeling entities such as users, products, accounts, merchants, or devices as nodes, and interactions such as purchases, reviews, transfers, or logins as edges [4]. This makes it possible to analyze connectivity patterns, neighborhoods, and multi-hop relationships that are difficult to express in tabular form [5]. As a result, graph-based learning methods, including Graph Neural Networks (GNNs), have become increasingly common in fraud detection tasks such as node classification, edge classification, and coordinated-group detection [6].

A knowledge graph (KG) extends this idea by attaching explicit semantics to nodes and edges. Instead of representing data as generic vertices and links, a knowledge graph uses typed entities and typed relationships, often together with node and edge properties [7]. This is especially important in fraud detection because suspicious behavior rarely appears as a single abnormal record; rather, it emerges as a pattern across multiple entity types and interaction types, such as coordinated review activity among users and products, many accounts sharing a device, or repeated transfers across related accounts [1]. By preserving this heterogeneous structure, knowledge graphs support richer relational modeling and provide a more meaningful substrate for downstream graph learning [8], [9].

A practical barrier to developing and evaluating graph-based fraud detection systems is the limited availability of realistic graph datasets. Privacy concerns, proprietary restrictions, and labeling cost often prevent researchers from accessing or sharing large fraud graphs. This makes controlled experimentation difficult, especially when the goal is to study how specific structural fraud patterns affect model behavior. To address this problem, this work explores the use of synthetic knowledge graphs generated directly from schema specifications and later corrupted with controlled fraud motifs.

In this work, the Amazon review domain is used as the reference setting for constructing and studying synthetic fraud graphs. This domain naturally forms a heterogeneous graph with users, reviews, products, and product groups, making it well suited to knowledge-graph based fraud modeling [10]–[13]. Prior work on review spam and group spam also provides concrete behavioral patterns, such as targeted products, repeated extreme ratings, and coordinated reviewer groups, that can be translated into graph-level fraud motifs [10], [14]. These properties make the Amazon review setting a suitable domain for studying both schema-driven graph generation and controlled fraud injection.

This paper presents a schema-driven synthetic KG framework for fraud detection in that setting. First, a clean synthetic review graph is generated from schema specifications inferred from the reference graph. Second, the clean synthetic graph is evaluated for structural fidelity using typed count agreement and typed degree-distribution similarity. Third, controlled user-level fraud motifs are injected into the graph to create corrupted review graphs with tunable suspicious behavior. Finally, these corrupted synthetic graphs are used to evaluate downstream user-level fraud detection with a heterogeneous GNN.

This capstone project is guided by the following objectives:

- **Synthetic knowledge graph generation:** Design and implement an end-to-end pipeline that generates synthetic knowledge graphs from schema specifications while preserving typed entities, typed relationships, and configurable structural constraints.

- **Controlled fraud injection:** Develop a reproducible fraud injection framework for synthetic review graphs that implants user-level fraudulent behavior through configurable reviewer–product motifs.
- **Review-domain evaluation:** Analyze the usefulness of the proposed synthetic-generation and fraud-injection pipeline by assessing their suitability for downstream graph-based fraud detection.

II. Related Works

A. Synthetic Knowledge Graphs

The term *synthetic knowledge graph* is used inconsistently in the literature. In some works, synthetic graphs are generated by learning from an existing real graph and then sampling new graphs that preserve structural or statistical properties [15]. In others, synthetic graphs are produced by augmenting an existing graph through perturbation, anonymization, or the addition of new structures [16].

Our focus is a *schema-driven* view of synthetic KG generation, where domain semantics are specified explicitly through entity types, relation types, property definitions, and structural constraints [17]. This perspective is most relevant to the present project because it supports controllability and reproducibility without requiring direct access to a large real fraud graph.

B. Synthetic Knowledge Graph Generation

Synthetic knowledge graph generation methods are often described as either *stochastic-based* or *semantic-based* [17]. Stochastic methods emphasize large-scale graph construction through random processes and statistical targets such as sparsity or degree distributions, but they usually do not encode domain semantics directly. In contrast, semantic or schema-driven methods generate typed graphs whose entities and relations are constrained by an explicit schema.

Our work is closest to the semantic-based line of research. The generator takes a user-defined schema containing node types, relationship types, property specifications, and per-relation cardinality constraints, and instantiates a graph that is both semantically valid and structurally configurable. This places the framework near prior schema-driven KG generators that aim to combine semantic validity with controllable structural realism [17]–[19].

C. Motif Injection in Graphs

A related idea to motif injection appears in planted-subgraph literature, where a structured local pattern is embedded into a larger background graph and later analyzed or detected [20]. In graph machine learning, a more practical analogue appears in subgraph-based backdoor attacks, where small trigger subgraphs are inserted into graphs to influence model behavior [21], [22].

The connection to motifs becomes more explicit in work such as *Motif-Backdoor*, which selects trigger structures according to motif statistics in the data [23]. Although these papers are framed as security attacks rather than fraud modeling, they remain relevant because they provide concrete examples of how localized substructures can be inserted into larger graphs. In contrast, our work uses such insertion in a constructive way, namely to implant controlled fraud-like patterns into synthetic review graphs.

D. Fraud Patterns in Review Graphs

Fraud in review systems is usually better understood as a behavioral pattern across reviewers, reviews, and products than as an isolated fake review. Lim et al. describe suspicious reviewers in terms of targeted products, targeted product groups, and repeated extreme rating behavior [10]. Mukherjee et al. further study coordinated spammer groups that jointly promote or demote products and characterize them through group-level signals such as support and deviation [14].

These findings motivate the use of localized reviewer–product motifs in our framework. Patterns such as same-product coordination, repeated extreme ratings, dense reviewer–product blocks, and overlapping reviewer neighborhoods are all graph-level counterparts of behaviors discussed in the review-spam literature. Therefore, this project focuses mainly on structural and rating-based fraud patterns rather than textual deception cues [24].

E. Knowledge Graphs and Heterogeneous GNNs for Fraud Detection

Recent fraud-detection research increasingly uses heterogeneous graphs and knowledge-rich graph structures instead of relying only on flat tabular features. A representative example is *xFraud*, which models fraud detection over a heterogeneous transaction graph and uses a heterogeneous GNN to learn from typed entities and typed relations [8]. Later work such as *MultiFraud* and *SEFraud* similarly reinforces the value of heterogeneous graph learning for fraud detection and explanation [25], [26].

The review domain has also seen graph-based anti-spam approaches in which reviewer, review, and product relations are used directly for downstream learning rather than only for hand-engineered behavioral features [27], [28]. This supports the modeling decision in the present work to represent review fraud at the graph level and use a heterogeneous GNN for user-level fraud classification.

Our work differs in emphasis from these prior studies. Rather than improving prediction on a fixed real-world graph, the goal here is to build a reusable schema-driven synthetic KG generation and corruption framework that can produce controlled review graphs for experimentation and downstream fraud detection.

III. Background

A. Knowledge Graphs

A knowledge graph is a directed graph composed of nodes and edges, where one node indicates an entity (a real object or abstract concept), and the edge between the two nodes conveys the semantic relation between the two entities [7].

In a knowledge graph each edge can be represented as a triple, (s, p, o), where s stands for subjects, o for objects and p for predicates. Subjects and objects are nodes that are called entities and predicate is the label of the edge connecting those nodes.

We define $E(G) = \{s|(s, _, _) \in G\} \cup \{o|(_, _, o) \in G\}$ as the set of entities in G and $P(G) = \{p|(_, p, _) \in G\}$ as the set of predicates in G.

$(u1, wrote, r1)$	$(r1, about, p1)$
$(u2, wrote, r2)$	$(r2, about, p1)$
$(u3, wrote, r3)$	$(r3, about, p2)$
$(p1, belongsto, electronics)$	$(p2, belongsto, home)$
$(p3, belongsto, books)$	

Fig. 1. Sample knowledge graph

Example 1. *An example of a knowledge graph can be taken from Figure III-A based on the Amazon Review Dataset 2023 [29]. If we were to compute E(G) and P(G) from the example, we would get*
$E(G) = \{u1, u2, u3, r1, r2, r3, p1, p2, p3, electronics, home, books\}$
$P(G) = \{wrote, about, belongsto\}$.

B. Power-Law

Many real-world networks (e.g., the Web, citation graphs, social networks) are highly heterogeneous: most nodes have only a few connections, while a small number of nodes act as hubs with very large degree [30]. Barabási and Albert argued that this heterogeneity is not well captured by classical random graph models (like Erdős–Rényi), which tend to produce degree distributions concentrated around an average value [30]. Instead, they observed that a wide range of empirical networks exhibit a power-law degree distribution, often summarized as:

$$P(k) \propto k^{-\gamma}$$

where $P(k)$ is the probability that a randomly selected node has degree k and γ is a positive constant (typically around 3 in many observed systems). A power law implies that high-degree nodes are rare but not exponentially suppressed, so hubs remain statistically significant even in very large graphs.

C. Motifs

Following Rossi et al., a heterogeneous graph is written as

$$G = (V, E, \phi, \xi),$$

where V is the set of nodes, E is the set of edges, ϕ is the node-type mapping, and ξ is the edge-type mapping [31]. A typed graphlet of G is defined as a connected induced heterogeneous subgraph

$$H = (V', E', \phi', \xi'),$$

such that (V', E') is a graphlet of (V, E), while ϕ' and ξ' are the restrictions of ϕ and ξ to the subgraph [31]. In heterogeneous networks, these small typed local connectivity patterns may also be viewed as motifs; in fact, Rossi et al.'s earlier formulation refers to typed graphlets as heterogeneous network motifs [32]. Therefore, in the remainder of this work, we use the term motif informally to refer to such small typed local subgraphs, while the formal object is the typed graphlet defined above.

In this work, we use the term *fraud motif* informally to refer to such a spammer-group structure when represented as a small typed local subgraph. This terminology is a modeling choice of our framework, while the underlying fraud concept is grounded in the review-spam definition of spammer groups [14], [31].

D. Heterogeneous Graph Neural Networks

Graph neural networks learn node representations by repeatedly aggregating information from a node's neighborhood. Among the most widely used neighborhood-aggregation models is GraphSAGE, which learns an aggregation function over local neighbors instead of learning a separate embedding for every node [33]. This makes GraphSAGE especially attractive when node features are available and when one wants a model that can generalize beyond a single fixed embedding table.

In this work, the downstream fraud detector is implemented as a heterogeneous GraphSAGE model. The underlying clean or corrupted review graph contains multiple node types, namely `User`, `Review`, `Product`, and `ProductGroup`, as well as typed relations such as `WROTE`, `ABOUT`, and `BELONGS_TO`. The base GraphSAGE layers are therefore lifted to the heterogeneous setting so that message passing respects node and edge types rather than treating the graph as homogeneous. This is consistent with broader fraud-detection literature showing that heterogeneous graph structure provides useful supervision for downstream prediction tasks [8], [25], [26].

E. Graph Fidelity Metrics

We use structural aggrements at multiple levels to evalaute how close the generated synthetic graph to the reference graph. First, global graph size is compared through total node count and total edge count. Second, typed count agreement is measured through node counts by label and edge counts by relation type. Third, distributional agreement is measured through source and target degree distributions for each typed relation signature [17], [34], [35].

Count-based comparisions are summarised using relative error as below:

$$\mathrm{RelErr}(x, \hat{x}) = \frac{|\hat{x} - x|}{|x|}.$$

where x is original value and x_hat is regenarted value.

A smaller relative error indicates closer agreement between the generated graph and the reference graph.

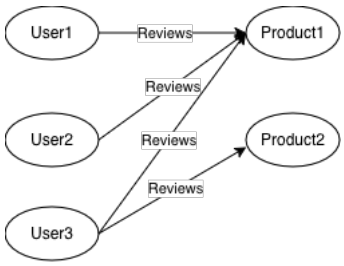

Fig. 2. Degree Distribution Example

Example 2. *Figure 2 also illustrates why degree distributions are useful for structural comparison. In the projected user–product view,* `User3` *reviews two products, while* `User1` *and* `User2` *each review one product, giving the user-side out-degree sequence* $[1, 1, 2]$. *On the product side,* `Product1` *receives three reviews and* `Product2` *receives one review, giving the product-side in-degree sequence* $[3, 1]$. *This shows that two graphs can have the same number of review edges but different structural concentration. Therefore, comparing degree distributions helps determine whether the synthetic graph preserves the local connectivity patterns of the original review graph.*

For distributional comparison, Jensen–Shannon (JS) divergence is used to measure the similarity between two empirical discrete distributions. Let P and Q denote the normalized degree histograms of the original and generated graphs for a fixed typed relation signature, and let

$$M = \frac{1}{2}(P + Q).$$

Then the Jensen–Shannon divergence is defined as

$$JS(P, Q) = \frac{1}{2}D_{KL}(P\|M) + \frac{1}{2}D_{KL}(Q\|M),$$

where

$$D_{KL}(P\|M) = \sum_{x\in\mathcal{X}} P(x) \log \frac{P(x)}{M(x)}.$$

Equivalently,

$$JS(P, Q) = H(M) - \frac{1}{2}H(P) - \frac{1}{2}H(Q),$$

where $H(\cdot)$ denotes Shannon entropy [36]. A smaller JS divergence indicates that the generated graph more closely matches the original degree distribution.

In the implemented evaluation pipeline, Laplace smoothing is applied to the reference histogram before expected counts are computed so that zero-frequency bins do not produce undefined statistics. Because node and edge counts may be matched closely by construction, the degree-distribution comparison is the more informative component of the structural evaluation.

F. Model Evaluation Metrics

The downstream user-fraud classifier is evaluated using precision, recall, F1-score, and area under the ROC curve (ROC-AUC). Let true positives, false positives, and false negatives be denoted by TP, FP, and FN, respectively. Precision is defined as

$$\text{Precision} = \frac{TP}{TP + FP},$$

and measures the fraction of predicted fraud users that are truly fraudulent. Recall is defined as

$$\text{Recall} = \frac{TP}{TP + FN},$$

and measures the fraction of all fraudulent users that are successfully detected. The F1-score is the harmonic mean of precision and recall,

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}},$$

and is especially useful when the classes are imbalanced [37].

ROC-AUC evaluates the ranking quality of the model across decision thresholds rather than only at one fixed threshold [38]. In the implemented pipeline, fraud probabilities are produced for user nodes, the classification threshold is selected on the validation split, and the resulting threshold is then used to compute precision, recall, and F1 on the train, validation, and test splits. This combination of threshold-dependent and threshold-independent metrics is important in fraud detection because one often wants to control the tradeoff between false positives and false negatives.

IV. Synthetic Graph Generation Framework

Since real fraud datasets are often limited by privacy, scarcity, and incomplete labels, a synthetic graph generation framework provides a practical alternative for systematic experimentation. Our framework generates domain-aware graphs from a schema specification, enabling the creation of entities, relations, and structural patterns that resemble real-world fraud environments. This section of the paper describes the schema specification and synthetic graph generation process with the help of *Amazon Review Dataset 2023* [29].

A. Schema Specification

At a high level, our schema is defined as a structured configuration that specifies the rules used to generate a synthetic graph [17], [18]. The schema consists of three main parts:

- global generator settings, such as the random seed and whether duplicate triples are allowed,
- node specifications, which define entity types, counts, and node attributes,
- relationship specifications, which define how node types are connected and what structural constraints those connections must satisfy.

This representation separates the description of the graph from the process of constructing it, making the framework flexible and reusable across different domains. The schema can be expressed either as a JSON configuration or as a pseudo-graph textual notation, where both representations encode the same node types, relationship types, property declarations, and structural constraints. In the following sections, we have represented the structure in terms of pseudo-graph format.

1) General Schema Structure:

```
@seed=<int>
@no_duplicate_triples=<true|false>

<NodeLabel1>(
    <property1>:<int>[min=<int>,max=<int>],
    <property2>:<string>,
    ...) [count=<int>]

<RelationType1>
    (<SourceNode> -> <TargetNode>)
        [count=<int>,
        allow_self_loop=<true|false>,
        from_degree.min=<int>,
        from_degree.max=<int>,
        to_degree.min=<int>,
        to_degree.max=<int>]
```

It is important to distinguish between *generator settings* and *relationship constraints*. Generator settings are global rules that affect how the entire graph is produced. For example, the `seed` controls reproducibility, ensuring that the same schema produces the same graph across runs, while `no_duplicate_triples` prevents repeated edges of the same type between the same source and destination nodes. In contrast, relationship constraints apply only to a specific edge type and control the valid structural patterns for that relation, such as whether self-loops are allowed and how many times nodes may participate in that relation.

2) Node Specifications: Under this general structure, the Amazon review dataset is represented using node types such as `User`, `Product`, `Review`, and `ProductGroup`, together with relationships that capture authorship, product association, and grouping.

Example 3. *`Review` node generated with multiple attributes, such as `review_id`, `rating`, `helpful_vote`, `images_count`, `text`, `title`, and `verified_purchase`, so a concrete instance may look like:*

```
Review(
  helpful_vote:int[min=0,max=259],
  images_count:int[min=0,max=18],
  rating:float[min=1.0,max=5.0],
  review_id:string[unique=true],
  text:string,
  title:string,
  verified_purchase:bool
) [count=130434]
```

3) Relationship Specifications: The relationship specification defines how generated nodes are connected, together with the structural rules that these connections must satisfy. In particular, the `from_degree` and `to_degree` fields specify degree constraints for a given relationship type. The `from_degree` field controls how many outgoing edges of that relationship a source node may have, while the `to_degree` field controls how many incoming edges of that relationship a destination node may have. The corresponding `min` and `max` values determine the allowable participation range for nodes on each side of the relation.

Example 4. *In the `WROTE` relationship, `from_degree: {min: 1, max: 341}` indicates that a `User` node can be connected to between 1 and 341 `Review` nodes through `WROTE` edges. At the same time, `to_degree: {min: 1, max: 1}` indicates that each `Review` node must receive exactly one incoming `WROTE` edge, ensuring that every review is authored by exactly one user. Similarly, in the `ABOUT` relationship, the degree constraints ensure that each review is associated with exactly one product, while a product may be associated with many reviews.*

```
WROTE(User -> Review)
[count=130434, allow_self_loop=false,
 from_degree.min=1, from_degree.max=341,
 to_degree.min=1, to_degree.max=1]
```

This specification means that each generated `WROTE` edge must connect a `User` node to a `Review` node, self-loops are not permitted, each review must be written by exactly one user, and each user may write between 1 and 341 reviews. A concrete edge instance may therefore take the following form:

```
(User) -[:WROTE]-> (Review)
```

In the same way, the `ABOUT` relationship ensures that each review refers to exactly one product, while allowing a product to be associated with many reviews. The `BELONGS_TO` relationship ensures that each product is assigned to exactly one product group, while each product group may contain multiple products. Together, these rules ensure that the generated graph is not only structurally valid, but also semantically aligned with the organization of the Amazon review domain. Internally, both schema representations are normalized to the same dictionary-based schema object before graph generation, so the choice between JSON and pseudo-graph notation affects usability and readability rather than the generation algorithm itself.

B. Synthetic Graph Generation

Given the schema specification, the synthetic graph generation process constructs a graph instance in three main stages: node generation, relationship degree allocation, and stub matching. The purpose of this staged design is to ensure that the final graph is not only valid with respect to the schema, but also structurally similar to real-world interaction graphs. In the Amazon review domain, this is especially important because review activity is naturally uneven: most users write only a small number of reviews, while a small number of highly active users write many reviews. Similarly, many products receive few reviews, while a smaller number of popular products accumulate many reviews. To capture this skewed behavior, the generator uses controlled degree allocation rather than uniform random edge creation.

a) Node generation.: The first stage creates node instances for each entity type defined in the schema. For each node type, the generator instantiates exactly the number of nodes specified by the `count` field, assigns the corresponding label, and generates property values according to the declared types and constraints. Unique properties such as `user_id`, `asin`, `review_id`, and `parent_asin` are generated so that no two nodes share the same value, while other attributes such as ratings, helpful votes, image counts, and Boolean flags are sampled from their allowed ranges. This stage establishes the full set of entities that will later participate in graph relationships.

b) Relationship degree allocation.: After generating the nodes, the framework determines how many times each node should participate in each relationship type. This is done using the relationship schema, which specifies the total number of edges to generate, together with the `from_degree` and `to_degree` constraints. The `from_degree` bounds determine how many outgoing edges a source node may have for a given relation, while the `to_degree` bounds determine how many incoming edges a destination node may have.

Rather than assigning these degrees uniformly at random, the generator may use a power-law based allocation for relations that are expected to be highly skewed. This is appropriate for many real-world graphs because participation is often concentrated in a small number of highly connected nodes. In the Amazon review setting, for example, the `WROTE` relation can be generated so that most users write only a few reviews, while a small number of users write many. Likewise, for the `ABOUT` relation, most products may be associated with few reviews, while a small number of popular products receive many reviews. Using a power-law guided degree allocation therefore helps the synthetic graph better reflect the heavy-tailed connectivity patterns commonly observed in practice.

At the same time, the allocation must still satisfy the schema constraints exactly. This means that the assigned degrees must remain within the minimum and maximum bounds for each node type, and the total number of allocated source participations must equal the total number of allocated destination participations, which must also equal the required edge count for that relationship. In this sense, the generator is not merely sampling degrees, but constructing a valid degree sequence that respects both the desired distributional pattern and the structural constraints of the schema.

c) Stub construction and matching.: Once the degree allocation is determined, the generator creates *stubs* for each relationship type. A stub is a temporary placeholder representing one endpoint of an edge, often viewed as a half-edge that is later paired with another stub to form a complete edge [39], [40]. A source stub corresponds to one outgoing participation slot of a source node, and a destination stub corresponds to one incoming participation slot of a destination node.

Example 5. *Let u be a user node and suppose its degree in the `WROTE` relation is fixed as*

$$d^{out}_{\texttt{WROTE}}(u) = 3.$$

Then u contributes three source stubs for that relation, which may be denoted by

$$s^{out}_1(u),\ s^{out}_2(u),\ s^{out}_3(u).$$

Now let r_1, r_2, r_3 be review nodes, and suppose each review must have exactly one incoming `WROTE` edge, that is,

$$d^{in}_{\texttt{WROTE}}(r_i) = 1, \qquad i \in \{1, 2, 3\}.$$

Then each review contributes one destination stub,

$$s^{in}(r_1),\ s^{in}(r_2),\ s^{in}(r_3).$$

A valid matching of source and destination stubs produces the edges

$$(u, r_1),\ (u, r_2),\ (u, r_3),$$

which means that user u wrote the three reviews r_1, r_2, r_3.

The generator then matches source stubs to destination stubs to form complete edges. This approach is useful because it separates two concerns: first deciding how often each node should participate in a relation, and then deciding which specific nodes should be connected. By doing so, the framework can enforce degree constraints systematically before the final edges are materialized. During matching, additional validity checks are applied. For example, if `allow_self_loop` is false, a node cannot be connected to itself for that relation. Similarly, if `no_duplicate_triples` is enabled, the same source node, relation type, and destination node combination cannot be created more than once.

d) Resulting graph structure.: For the Amazon schema, this procedure ensures that each `Review` node receives exactly one incoming `WROTE` edge from a `User` node and exactly one outgoing `ABOUT` edge to a `Product` node. Each `Product` node, in turn, has exactly one outgoing `BELONGS_TO` edge to a `ProductGroup`. At the same time, users may write multiple reviews, products may accumulate many reviews, and product groups may contain multiple products, all within the degree ranges defined by the schema. As a result, the generated graph is both structurally valid and more representative of realistic interaction patterns than a graph produced through unconstrained random linking.

Overall, this generation procedure transforms a declarative schema into a concrete graph instance through a sequence of controlled operations: creating nodes, allocating degree participation, and matching stubs into valid edges. This design improves reproducibility, interpretability, and realism, while making it easier to adapt the framework to new domains and to later inject domain-specific fraud patterns in a structured way.

C. Validation of the Clean Synthetic Graphs

Before applying fraud injection, the generated synthetic graphs were evaluated against the original Amazon review graph from which the schema was inferred. The purpose of this step was to verify that the schema-driven generator preserved the typed structural properties of the reference graph prior

TABLE I
DEGREE-DISTRIBUTION SIMILARITY BETWEEN THE ORIGINAL GRAPH AND THE CLEAN SYNTHETIC GRAPHS, MEASURED USING JENSEN–SHANNON DIVERGENCE.

Relation signature	Small	Medium	Large
WROTE source JS	0.0068	0.0054	0.0071
WROTE target JS	0.0000	0.0000	0.0000
ABOUT source JS	0.0000	0.0000	0.0000
ABOUT target JS	0.0102	0.1205	0.4643
BELONGS_TO source JS	0.0000	0.0000	0.0000
BELONGS_TO target JS	0.00002	0.000006	0.000004

to any corruption. This follows common practice in graph-generation research, where the quality of a generated graph is assessed by comparing its structural statistics with those of a reference graph [17], [34], [35].

The validation was conducted on three clean synthetic graphs of different scales, denoted *small*, *medium*, and *large*. Global and typed count agreement was exact in our setting, with zero relative error across node and edge counts because these quantities were fixed by the schema. The more informative comparison was therefore the typed degree-distribution agreement for each principal relation signature, examined through both source-degree and target-degree behavior. Table I reports this similarity using Jensen–Shannon divergence [36].

As expected, count agreement was strongest for graph statistics that were explicitly constrained by the schema, while the more informative comparison was the degree-distribution similarity of each typed relation. Lower Jensen–Shannon divergence indicates that the generated graph more closely matched the original relation-level connectivity patterns. These clean-graph validation results provide the structural baseline for the later fraud-injection and downstream fraud-detection experiments.

V. FRAUD INJECTION FRAMEWORK

Once a clean synthetic review graph has been generated from the schema, the next stage is to corrupt it in a controlled and reproducible manner with user-level fraudulent behavior. The purpose of this stage is not merely to add noise, but to implant structured patterns of suspicious activity that resemble behaviors reported in the review-spam literature. In this way, the final graph contains both benign and fraudulent users, together with realistic reviewer–product interaction patterns that can later be used for fraud detection experiments.

A. User-Level Fraud Formulation

In this work, fraud is modeled at the *user level*. That is, the prediction target is whether a user is fraudulent or benign, rather than whether an individual review is fraudulent. This choice reflects the intuition that fraud in review systems is often a behavioral property of an account, expressed through the collection of reviews it writes and the products it targets.

Formally, let the generated knowledge graph be denoted by

$$G = (V, E),$$

where V contains nodes of type `User`, `Review`, `Product`, and `ProductGroup`, and E contains the typed relations `WROTE`, `ABOUT`, and `BELONGS_TO`. A subset of user nodes

$$U_f \subseteq V_{\texttt{User}}$$

is selected as the *criminal pool*. Each user $u \in U_f$ is labeled with `fraud = true`, while all other users remain unlabeled or benign. Review nodes themselves are not assigned fraud labels. Instead, fraud is represented indirectly through the structural and rating behavior induced by users in U_f.

B. Fraud Motifs in Review Graphs

The corruption stage is based on the idea that review fraud can be represented as a collection of small, repeated structural patterns, or *fraud motifs*, implanted into an otherwise clean graph. A fraud motif is a localized reviewer–product configuration whose structure and rating behavior reflect suspicious activity described in prior review-spam studies [10], [14]. Modeling fraud in this way makes the corruption process explicit, controllable, and interpretable.

The framework currently supports the following motif families:

- **Group same-product motif:** multiple fraudulent users review the same product with aligned extreme ratings. This captures coordinated promotion or demotion of a target product.
- **Single-user many-products motif:** one fraudulent user reviews many products, often within the same product group, with repeated rating behavior. This captures concentrated targeting behavior by an individual account.
- **Dense bipartite motif:** a subset of fraudulent users jointly targets a subset of products, producing a dense reviewer–product interaction block. This models coordinated multi-target campaigns.
- **Near-duplicate neighborhood motif:** several fraudulent users review highly overlapping sets of products, producing similar local neighborhoods in the graph. This models repeated collaboration among the same suspicious actors.
- **Repeated behavior motif:** a fraudulent user, or a small fraudulent user subset, repeatedly gives the same extreme ratings across many products. This captures unnaturally consistent rating behavior over time and across targets.

These motifs are inspired by prior work on rating-behavior spam and group spam. In our framework, these ideas are translated into graph-level corruption patterns that can be added systematically to the synthetic graph.

Example 6. *Figure 3 provides an example of a dense reviewer–product fraud motif block. Suppose three users, u_1, u_2, and u_3, all review the same two products, p_1 and p_2. In the review graph, this creates repeated paths of the form $u_i \xrightarrow{\texttt{WROTE}} r_{ij} \xrightarrow{\texttt{ABOUT}} p_j$, where each suspicious user writes reviews about the same small set of products. This structure represents coordinated behavior because the users share unusually similar product neighborhoods. During fraud injection, the selected users are labeled as fraudulent, while*

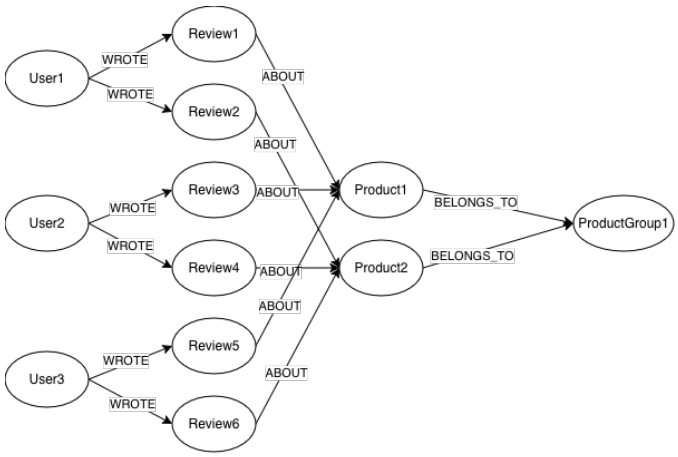

Fig. 3. Dense Reviewer-Product Block Motif Example

the generated review and product connections create the structural evidence used by the downstream GNN model.

C. Criminal Pool Construction

We define a *criminal pool* as a subset of user nodes chosen to participate in the injected suspicious activity. The size of this pool is controlled by user-defined parameters, including a minimum number of fraudulent users and a minimum fraction of the total user population. This allows the corruption process to scale with the graph size while maintaining flexibility across domains and experimental settings.

Let N_R denote the number of existing review nodes in the clean graph, and let $\rho \in (0, 1)$ denote the requested corruption rate. The framework first computes a malicious review budget

$$B = \rho \cdot N_R,$$

which determines how many suspicious reviews should be added. The criminal pool size is then chosen so that it is large enough to support the requested budget without forcing every fraudulent user to behave unrealistically aggressively.

An important design choice is that fraudulent users are not always selected independently for each injected campaign. Instead, users that have already participated in previous suspicious motifs may be sampled again with higher probability. This reuse mechanism allows some fraudulent users to appear repeatedly across multiple campaigns, producing more realistic recurring offenders rather than isolated one-time anomalies.

D. Injection Operator

After the criminal pool has been defined, the framework applies a motif injection operator that implants fraudulent reviewer–product patterns into the clean host graph. The corruption process is *injection-based*: it adds new review activity on top of the existing graph while preserving the original benign structure. This is preferable to replacement-based corruption because it maintains the domain-faithful graph that was produced by the schema-driven generator, while overlaying controlled suspicious subgraphs on top of it.

At a high level, the injection procedure repeats the following steps until the malicious review budget is exhausted:

1) Select a fraud motif family according to user-defined pattern weights.
2) Sample one or more users from the criminal pool.
3) Select one or more target products, optionally favoring products from the same product group or products with relatively low degree.
4) Create new `Review` nodes with suspiciously aligned or repeated rating behavior.
5) Connect the new reviews into the graph using

```
(User) -[:WROTE]-> (Review)
(Review) -[:ABOUT]-> (Product)
```

This operation can be interpreted as a form of motif injection, where a small structured reviewer–product pattern is inserted into a larger graph while leaving the rest of the host graph intact. In this work, motif injection is not used for adversarial model attacks, but rather as a principled graph-construction mechanism for building controlled fraud benchmarks.

E. Corruption Control and Parameterization

A key strength of the proposed framework is that fraud injection is explicitly parameterized. Rather than generating a single fixed corruption pattern, the user can control the amount and style of corruption through configurable parameters such as:

- **corruption rate**, which determines the overall malicious review budget,
- **criminal pool size**, which determines how many users are available to participate in fraud,
- **pattern weights**, which determine how often each motif family is selected,
- **reuse bias**, which controls how strongly the same fraudulent users are reused across campaigns,
- **rating tendencies**, which influence whether a user or campaign behaves more like a promoter or a demoter.

These parameters make it possible to generate families of corrupted graphs under different levels of class imbalance, different mixtures of fraud motifs, and different degrees of concentration or dispersion. As a result, the framework supports controlled experimentation rather than a single ad hoc corruption strategy.

F. Benign Activity Blending

If fraudulent users only ever participate in suspicious motifs, they may become trivially separable from benign users. To avoid this, the framework optionally adds additional ordinary-looking review activity by users in the criminal pool. These additional reviews are designed to resemble benign behavior more closely, for example by assigning ratings closer to product-level averages and by spreading activity across less concentrated targets.

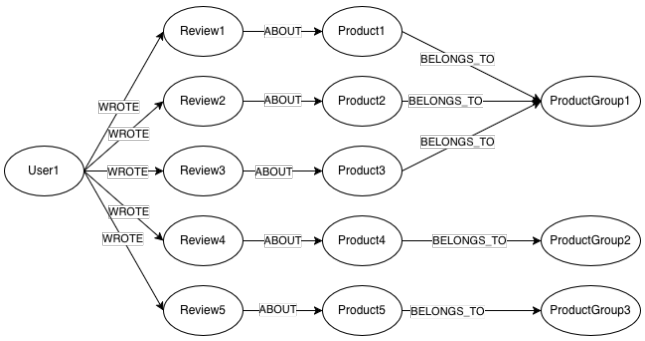

Fig. 4. Benign activity blending example. `Review1`–`Review3` target products in `ProductGroup1`, representing a concentrated defamation pattern, while `Review4`–`Review5` are spread across other product groups to resemble benign behavior.

Example 7. *Figure 4 shows an example of benign activity blending for one fraudulent user. In this example,* `User1` *writes five reviews. The first three reviews represent the suspicious part of the behavior. Here,* `Review1`*,* `Review2`*, and* `Review3` *all target products that belong to* `ProductGroup1`*. If these reviews contain low or extreme ratings, the pattern can be interpreted as a concentrated attempt to demote or defame that product group. The suspicious behavior is therefore not only that the same user writes multiple reviews, but that several of those reviews are concentrated within the same product group.*

The remaining reviews act as benign-looking activity, these reviews are spread across different product groups and can be assigned more ordinary rating behavior. This makes the fraudulent user appear less obviously suspicious because the defamation activity is mixed with normal-looking review behavior. The user is still labeled as fraudulent, but the graph neighborhood now contains both suspicious and benign-looking interactions. As a result, the downstream GNN must learn from the user's overall interaction pattern rather than relying only on one isolated suspicious structure.

This blending step does not change the fact that the user belongs to the fraudulent class, but it reduces the extent to which fraud users can be identified through a single obvious pattern. Consequently, downstream fraud-detection models must rely more on the overall behavioral signature of a user rather than on one isolated extreme event. This design is consistent with the broader literature on review fraud, which suggests that suspicious behavior often emerges from a user's aggregate interaction pattern rather than from any one review in isolation [10], [24].

G. Resulting Corrupted Graph

After injection, the final graph contains a mixture of benign users and fraudulent users, together with both ordinary and suspicious review activity. The graph remains schema-consistent because every injected review is still represented as a valid `Review` node connected to exactly one author and exactly one product. At the same time, the inserted motifs create localized reviewer–product patterns that mimic behaviors reported in the review-spam literature.

The final corrupted graph therefore serves as a controlled benchmark for user-level fraud detection. It preserves the semantic structure of the original knowledge graph, supports tunable corruption levels, and makes the source of suspicious behavior explicit through the design of the injected motifs.

VI. Evaluation

A. Experimental Setup

The prediction task in this work is user-level fraud detection. User nodes selected into the criminal pool during fraud injection are labeled as fraudulent, while all remaining user nodes are treated as benign. Review, product, and product-group nodes remain part of the graph as contextual structure, but they are not themselves prediction targets. The model used is a heterogeneous GraphSAGE classifier that performs message passing over the typed review graph and outputs fraud probabilities for user nodes [33].

The implemented pipeline proceeds in four stages. First, a clean heterogeneous review graph is generated from a schema inferred from the original Amazon review graph. Second, a sanitization step removes strongly suspicious naturally occurring user patterns so that the starting synthetic graph is less contaminated by pre-existing anomalies. Third, a controlled fraud-injection process implants user-level fraud motifs into the graph, including same-product coordination, repeated extreme rating behavior, dense reviewer–product blocks, and highly overlapping reviewer neighborhoods. Fourth, the resulting corrupted graph is converted into a heterogeneous graph-learning representation and used to train the user-fraud classifier. The training pipeline uses class-weighted cross-entropy, validation-based threshold selection, and evaluation on train, validation, and test splits using precision, recall, F1-score, and ROC-AUC [37], [38].

B. Fraud Detection on Synthetic Graphs of Different Sizes

To study the usefulness of the synthetic graph as a training substrate, experiments were conducted on three synthetic graphs of different scales, denoted *small*, *medium*, and *large*. Across these settings, the schema structure, relation semantics, and fraud-injection logic were kept fixed, while the graph size was varied by scaling the counts of nodes and typed relations. This design makes it possible to study how the fraud-detection performance changes as the amount of synthetic graph structure available for training increases.

For each synthetic graph, the user nodes were divided into train, validation, and test splits, and the heterogeneous GraphSAGE model was trained only on the synthetic graph. Model selection and threshold tuning were performed on the validation split. Table II reports the resulting precision, recall, F1-score, and ROC-AUC for the three synthetic graph sizes. These results assess whether larger synthetic graphs improve the learned user representations and whether the proposed fraud motifs remain learnable across scales.

TABLE II
USER-FRAUD DETECTION RESULTS WHEN TRAINING AND TESTING

Graph size	Precision	Recall	F1	AUC
Small	0.8615	0.6829	0.7619	0.8791
Medium	0.9879	0.7417	0.8473	0.8787
Large	0.9846	0.7494	0.8510	0.8846

Overall, the synthetic-only experiments evaluate whether the generated and corrupted graphs are useful not just as structurally plausible graphs, but also as training data for a fraud detector. In particular, they test whether the schema-driven generation process and the injected fraud motifs provide enough relational signal for a heterogeneous GNN to separate fraudulent users from benign users.

C. Discussion

The results suggest that the proposed synthetic-generation and fraud-injection pipeline produces graphs that are useful for downstream user-level fraud detection. As graph size increases, the model generally performs better, which indicates that larger synthetic graphs provide richer relational structure and more learnable fraud patterns for the heterogeneous GNN.

At the same time, the improvement is not uniform across scales. The main performance gain appears when moving beyond the smallest graph, while the difference between the larger settings is more modest. This suggests that increasing graph size is beneficial, but that the benefit begins to level off once the synthetic graph already contains sufficient structural variety and enough injected fraud behavior for effective learning.

Another notable pattern is that the model appears more successful at producing reliable fraud predictions than at recovering all fraudulent users. In other words, the learned representations support strong discrimination between suspicious and benign users, but some fraudulent users remain harder to identify. This indicates that the current framework already provides meaningful fraud-learning signal, while also leaving room for improvement in capturing a broader range of fraudulent behavior.

Taken together, these findings support the usefulness of the framework in two ways. First, the clean-graph validation shows that the generated graph remains structurally aligned with the reference graph. Second, the downstream experiments show that the injected fraud motifs are learnable and remain effective across different graph sizes. Overall, this suggests that the framework is not only structurally meaningful, but also practically useful as a controlled environment for graph-based fraud-detection experiments.

VII. CONCLUSION

This work presented a schema-driven synthetic knowledge graph framework for fraud detection in the Amazon review domain. The framework generates clean synthetic review graphs from schema specifications that preserve typed entities, typed relations, and structural constraints, and then corrupts those graphs through controlled user-level fraud motifs. In this way, the framework provides a systematic way to construct review graphs that are both semantically meaningful and suitable for controlled fraud-detection experiments.

The evaluation showed that the clean synthetic graphs remained structurally aligned with the reference graph, particularly for graph properties that were explicitly constrained by the schema, while the degree-distribution comparisons provided a more informative view of structural similarity. The downstream experiments further showed that the corrupted synthetic graphs provide useful relational signal for user-level fraud detection with a heterogeneous GNN, and that performance generally improves as larger synthetic graphs are used for training. Together, these findings suggest that the proposed framework is not only capable of generating structurally plausible review graphs, but also useful as a practical environment for studying graph-based fraud detection under controlled conditions.

Overall, the main contribution of this project is the combination of schema-driven synthetic graph generation, controlled fraud injection, structural fidelity evaluation, and downstream fraud-detection assessment within a single pipeline. This makes the framework useful for experimentation in settings where real fraud graph data is limited, difficult to share, or costly to label. Future work can extend this framework by exploring a wider range of corruption settings, incorporating temporal information so that injected fraud patterns better reflect realistic activity timing, generating context-aware string attributes rather than random text values, expanding the diversity of fraud motifs, and improving the ability of the downstream model to recover harder-to-detect fraudulent users.

REFERENCES

[1] T. Pourhabibi, K.-L. Ong, B. H. Kam, and Y. L. Boo, "Fraud detection: A systematic literature review of graph-based anomaly detection approaches," *Decision Support Systems*, vol. 133, p. 113303, 2020.

[2] A. Zhou, X. Xu, R. Raghunathan, A. Lal, X. Guan, B. Yu, and B. Li, "Knowgraph: Knowledge-enabled anomaly detection via logical reasoning on graph data," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 168–182.

[3] E. Li, M. Chen, S. Xiang, and L. Chen, "Graph learning-empowered financial fraud detection: Progress and future directions," *Intelligent Computing*, vol. 4, p. 0146, 2025.

[4] R. Angles, "The property graph database model." in *AMW*, 2018.

[5] L. Akoglu, H. Tong, and D. Koutra, "Graph based anomaly detection and description: a survey," *Data mining and knowledge discovery*, vol. 29, no. 3, pp. 626–688, 2015.

[6] D. Cheng, Y. Zou, S. Xiang, and C. Jiang, "Graph neural networks for financial fraud detection: a review," *Frontiers of Computer Science*, vol. 19, no. 9, p. 199609, 2025.

[7] C. Peng, F. Xia, M. Naseriparsa, and F. Osborne, "Knowledge graphs: Opportunities and challenges," *Artificial intelligence review*, vol. 56, no. 11, pp. 13 071–13 102, 2023.

[8] S. X. Rao, S. Zhang, Z. Han, Z. Zhang, W. Min, Z. Chen, Y. Shan, Y. Zhao, and C. Zhang, "xfraud: explainable fraud transaction detection," *arXiv preprint arXiv:2011.12193*, 2020.

[9] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. Van Den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *European semantic web conference*. Springer, 2018, pp. 593–607.

[10] E.-P. Lim, V.-A. Nguyen, N. Jindal, B. Liu, and H. W. Lauw, "Detecting product review spammers using rating behaviors," in *Proceedings of the 19th ACM international conference on Information and knowledge management*, 2010, pp. 939–948.

[11] N. Jindal, B. Liu, and E.-P. Lim, "Finding unusual review patterns using unexpected rules," in *Proceedings of the 19th ACM International Conference on Information and Knowledge Management*, 2010, pp. 1549–1552.

[12] Y. Lu, L. Zhang, Y. Xiao, and Y. Li, "Simultaneously detecting fake reviews and review spammers using factor graph model," in *Proceedings of the 5th Annual ACM Web Science Conference*, 2013, pp. 225–233.

[13] S. He, B. Hollenbeck, G. Overgoor, D. Proserpio, and A. Tosyali, "Detecting fake-review buyers using network structure: Direct evidence from amazon," *Proceedings of the National Academy of Sciences*, vol. 119, no. 47, p. e2211932119, 2022.

[14] A. Mukherjee, B. Liu, and N. Glance, "Spotting fake reviewer groups in consumer reviews," in *Proceedings of the 21st international conference on World Wide Web*, 2012, pp. 191–200.

[15] P. K. Choubey, X. Su, M. Luo, X. Peng, C. Xiong, T. Le, S. Rosenman, V. Lal, P. Mui, R. Ho *et al.*, "Distill-synthkg: Distilling knowledge graph synthesis workflow for improved coverage and efficiency," *arXiv preprint arXiv:2410.16597*, 2024.

[16] T. John, E. B. Johnsen, and E. Kamburjan, "Rdfmutate: Mutation-based generation of knowledge graphs," in *International Semantic Web Conference*. Springer, 2025, pp. 295–312.

[17] Z. Feng, W. Mayer, K. He, S. Kwashie, M. Stumptner, G. Grossmann, R. Peng, and W. Huang, "A schema-driven synthetic knowledge graph generation approach with extended graph differential dependencies (gdd x s)," *IEEE Access*, vol. 9, pp. 5609–5639, 2020.

[18] G. Bagan, A. Bonifati, R. Ciucanu, G. H. Fletcher, A. Lemay, and N. Advokaat, "gmark: Schema-driven generation of graphs and queries," *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 4, pp. 856–869, 2016.

[19] N. Hubert, P. Monnin, M. d'Aquin, D. Monticolo, and A. Brun, "Pygraft: Configurable generation of synthetic schemas and knowledge graphs at your fingertips," in *European Semantic Web Conference*. Springer, 2024, pp. 3–20.

[20] D. Elimelech and W. Huleihel, "Detecting arbitrary planted subgraphs in random graphs," *arXiv preprint arXiv:2503.19069*, 2025.

[21] Z. Zhang, J. Jia, B. Wang, and N. Z. Gong, "Backdoor attacks to graph neural networks," in *Proceedings of the 26th ACM Symposium on Access Control Models and Technologies*, 2021, pp. 15–26.

[22] Z. Xi, R. Pang, S. Ji, and T. Wang, "Graph backdoor," in *30th USENIX security symposium (USENIX Security 21)*, 2021, pp. 1523–1540.

[23] H. Zheng, H. Xiong, J. Chen, H. Ma, and G. Huang, "Motif-backdoor: Rethinking the backdoor attack on graph neural networks via motifs," *IEEE Transactions on Computational Social Systems*, vol. 11, no. 2, pp. 2479–2493, 2024.

[24] A. Mukherjee, V. Venkataraman, B. Liu, and N. Glance, "What yelp fake review filter might be doing?" in *Proceedings of the international AAAI conference on web and social media*, vol. 7, no. 1, 2013, pp. 409–418.

[25] B. Wu, K.-M. Chao, and Y. Li, "Heterogeneous graph neural networks for fraud detection and explanation in supply chain finance," *Information Systems*, vol. 121, p. 102335, 2024.

[26] K. Li, T. Yang, M. Zhou, J. Meng, S. Wang, Y. Wu, B. Tan, H. Song, L. Pan, F. Yu, Z. Sheng, and Y. Tong, "Sefraud: Graph-based self-explainable fraud detection via interpretative mask learning," *arXiv preprint arXiv:2406.11389*, 2024.

[27] A. Li, Z. Qin, R. Liu, Y. Yang, and D. Li, "Spam review detection with graph convolutional networks," in *Proceedings of the 28th ACM international conference on information and knowledge management*, 2019, pp. 2703–2711.

[28] S. Yu, J. Ren, S. Li, M. Naseriparsa, and F. Xia, "Graph learning for fake review detection," *Frontiers in artificial intelligence*, vol. 5, p. 922589, 2022.

[29] Y. Hou, J. Li, Z. He, A. Yan, X. Chen, and J. McAuley, "Bridging language and items for retrieval and recommendation," *arXiv preprint arXiv:2403.03952*, 2024.

[30] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *science*, vol. 286, no. 5439, pp. 509–512, 1999.

[31] R. A. Rossi, N. K. Ahmed, A. Carranza, D. Arbour, A. Rao, S. Kim, and E. Koh, "Heterogeneous graphlets," *ACM Transactions on Knowledge Discovery from Data (TKDD)*, vol. 15, no. 1, pp. 1–43, 2020.

[32] ——, "Heterogeneous network motifs," *arXiv preprint arXiv:1901.10026*, 2019.

[33] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[34] A. Bojchevski, O. Shchur, D. Zügner, and S. Günnemann, "Netgan: Generating graphs via random walks," in *International conference on machine learning*. PMLR, 2018, pp. 610–619.

[35] J. You, R. Ying, X. Ren, W. Hamilton, and J. Leskovec, "Graphrnn: Generating realistic graphs with deep auto-regressive models," in *International conference on machine learning*. PMLR, 2018, pp. 5708–5717.

[36] J. Lin, "Divergence measures based on the shannon entropy," *IEEE Transactions on Information Theory*, vol. 37, no. 1, pp. 145–151, 1991.

[37] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.

[38] T. Fawcett, "An introduction to roc analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[39] B. K. Fosdick, D. B. Larremore, J. Nishimura, and J. Ugander, "Configuring random graph models with fixed degree sequences," *SIAM Review*, vol. 60, no. 2, pp. 315–355, 2018.

[40] G. Casiraghi and V. Nanumyan, "Configuration models as an urn problem," *Scientific Reports*, vol. 11, no. 1, p. 13416, 2021.